

```

// =====
// WEATHER GATE v5.2 - DUAL CORE FINAL (исправленный)
// ESP32 WT32-ETH01 + Ebyte E07-900M10S + DS3231 + BMP280
// Исправления: устранён конфликт RadioLib, race condition, оптимизация очереди
// =====

#define MQTT_MAX_PACKET_SIZE 1024

#include <ETH.h>
#include <WiFi.h>
#include <Wire.h>
#include <RTCLib.h>
#include <SPI.h>
#include <LittleFS.h>
#include <ArduinoJson.h>
#include <WebServer.h>
#include <PubSubClient.h>
#include <Preferences.h>
#include <HTTPClient.h>
#include <HTTPUpdate.h>
#include <esp_task_wdt.h>
#include <time.h>
#include <math.h>
#include <rtl_433_ESP.h>
#include <RadioLib.h>
#include <Adafruit_Sensor.h>
#include <Adafruit_BMP280.h>

extern CC1101 radio;

// ===== КОНФИГУРАЦИЯ =====
SET_LOOP_TASK_STACK_SIZE(20 * 1024);
#define CURRENT_VERSION "5.2"
#define FW_NAME "weather_gate"
#define TIMEZONE_OFFSET 7

// ===== ПИНЫ (В СООТВЕТСТВИИ С БИБЛИОТЕКОЙ) =====
// SPI для Ebyte E07-900M10S (пины уже заданы в rtl_433_esp.h)
// CS=2, GDO0=39, SCK=12, MISO=15, MOSI=14
#define PIN_GDO0 39

// I2C Канал 0 (DS3231 Часы) - родные пины
#define PIN_I2C0_SDA 32
#define PIN_I2C0_SCL 33

```

```

// I2C Канал 1 (BMP280 Давление) - свободные пины
#define PIN_I2C1_SDA 5
#define PIN_I2C1_SCL 4

// ===== НАСТРОЙКИ =====
#define POINTS_HISTORY 288
#define HISTORY_INTERVAL 300000
#define MQTT_PUBLISH_INTERVAL 60000
#define MQTT_RECONNECT_INTERVAL 10000
#define SAVE_INTERVAL 3600000 // 1 час
#define HA_PORT 8123
#define HA_OTA_PATH "/local/ota/weather_gate/"
#define MQTT_TOPIC_WEATHER_DEFAULT "weather_gate/data"
#define BMP_READ_INTERVAL 30000

// ===== ПАРАМЕТРЫ СКАНЕРА =====
const int SCAN_STEPS = 21;
const float SCAN_START_FREQ = 914.80;
const float SCAN_STEP = 0.02;
const unsigned long SCAN_STEP_TIME = 65000;

// ===== УСРЕДНЕНИЕ ВЕТРА =====
const int WIND_AVG_SAMPLES = 10;
const int WIND_SPEED_SAMPLES = 10;

// ===== ZAMBRETTI =====
const int PRESSURE_HISTORY_SIZE = 12;

// ===== СТРУКТУРА ДЛЯ ОЧЕРЕДИ =====
struct WeatherMessage {
    uint8_t station_id;
    bool battery_ok;
    float outdoor_temp;
    uint8_t outdoor_humidity;
    uint16_t wind_dir_deg;
    float wind_speed_ms;
    float wind_gust_ms;
    float rain_mm;
    float current_freq;
    int8_t rssi;
    bool data_valid;
};

```

```
// ===== ГЛОБАЛЬНЫЕ ПЕРЕМЕННЫЕ =====
```

```
QueueHandle_t weatherQueue = NULL;
```

```
// I2C объекты (разные шины)
```

```
TwoWire I2C_Clock = TwoWire(0);
```

```
TwoWire I2C_Bmp = TwoWire(1);
```

```
RTC_DS3231 rtc;
```

```
Adafruit_BMP280 bmp(&I2C_Bmp);
```

```
// Объект библиотеки rtl_433
```

```
rtl_433_ESP rf;
```

```
// Текущая частота (доступна обоим ядрам)
```

```
volatile float current_working_frequency = 915.00;
```

```
// Флаги
```

```
bool eth_connected = false;
```

```
bool mqtt_connected = false;
```

```
bool discovery_sent = false;
```

```
bool scan_active = false;
```

```
// Метеоданные (глобальные копии на Core 1)
```

```
float temperature = 0, humidity = 0, pressure = 1013.25;
```

```
float wind_avg_smooth = 0, wind_gust_max = 0;
```

```
int wind_dir_avg = 0;
```

```
float rain = 0;
```

```
float last_rssi = -120.0;
```

```
int sensor_id = -1;
```

```
String outdoor_battery = "OK";
```

```
// Статистика
```

```
float temp_min = 0, temp_max = 0;
```

```
float pressure_min = 1013.25, pressure_max = 1013.25;
```

```
float wind_max = 0, rain_max = 0;
```

```
// Настройки
```

```
String hostname = "weather_gate";
```

```
String mqtt_ip = "";
```

```
String mqtt_user = "";
```

```
String mqtt_password = "";
```

```
String mqtt_topic_weather = MQTT_TOPIC_WEATHER_DEFAULT;
```

```
float bmp_calibration_offset = 31.7;
```

```
int altitude_meters = 150;
```

```
float current_freq = 915.00;
```

// Счётчики

```
uint32_t radio_errors = 0, radio_success = 0;  
uint32_t lastSaveTime = 0, lastHistoryTime = 0;  
uint32_t lastMQTTReconnect = 0, lastMQTTPublish = 0;
```

// История

```
struct {  
    float temp[POINTS_HISTORY];  
    float pressure[POINTS_HISTORY];  
    float wind[POINTS_HISTORY];  
    float rain[POINTS_HISTORY];  
    int head = 0;  
} history;
```

// Zambretti

```
float pressure_history[PRESSURE_HISTORY_SIZE];  
int pressure_history_idx = 0;  
bool pressure_history_full = false;  
String zambretti_forecast = "Ожидание данных...";  
String pressure_trend_desc = "Стабильно";
```

// Усреднение ветра

```
float wind_dir_history_sin[WIND_AVG_SAMPLES];  
float wind_dir_history_cos[WIND_AVG_SAMPLES];  
int wind_sample_idx = 0;  
bool wind_history_filled = false;  
float wind_speed_history[WIND_SPEED_SAMPLES];  
int wind_speed_idx = 0;  
bool wind_speed_filled = false;
```

// Объекты

```
WebServer server(80);  
Preferences pref;  
WiFiClient ethClient;  
PubSubClient mqtt(ethClient);  
char msgBuffer[512];
```

// BMP280 переменные

```
float bmp_pressure_raw = 1013.25;  
float bmp_temperature = 0;  
unsigned long lastBMPRead = 0;
```

// Сканер

```

float scan_results[SCAN_STEPS];
int scan_current_step = 0;
float scan_best_freq = 915.00;
float scan_best_rssi = -150.0;
float scan_step_max_rssi = -150.0;
unsigned long scan_step_start = 0;
float scan_instant_rssi = -120.0;

// ===== ПРОТОТИПЫ =====

void loadSettings();
void saveAll();
void saveCounters();
void periodicSave();
void printResetReason();
String formatDate(DateTime dt);
float parseSafeFloat(String val);
bool handleFileRead(String path);
void addHistory(float t, float p, float w, float r);
void updatePressureHistory(float currentPressure);
float calculatePressureTrend();
void calculateZambrettiForecast();
void updateWindAverage(int degrees);
void updateWindSpeedAverage(float current_speed, float current_gust);
String getWindDirectionText(int degrees);
void registerAllSensors();
void sendDiscovery(const char* name, const char* unit, const char* dev_cla, const char* icon = nullptr);
void publishMQTTData();
void broadcastMQTTData(); // форсированная отправка после получения пакета
void reconnectMQTT();
void mqttCallback(char* topic, byte* payload, unsigned int length);
bool getUpdateInfo(UpdateInfo &info);
int compareVersions(String newVer, String curVer);
void performOTA();
void rtl433_Callback(char* message);
void radioTask(void *pvParameters);
float runHardwareScanner();
void updateAutoScanner();
void readBMP280();

struct UpdateInfo {
String version;
String fw_url;
String fs_url;
String fw_md5;

```

```
String fs_md5;
String changelog;
};
```

```
// ===== ДИАГНОСТИКА =====
```

```
void printResetReason() {
    esp_reset_reason_t reason = esp_reset_reason();
    Serial.printf("[%08lu] [SYS] === RESET DIAGNOSTIC ===\n", millis());
    switch(reason) {
        case ESP_RST_POWERON: Serial.println("[SYS] Reset: POWER_ON"); break;
        case ESP_RST_EXT: Serial.println("[SYS] Reset: EXTERNAL"); break;
        case ESP_RST_SW: Serial.println("[SYS] Reset: SOFTWARE"); break;
        case ESP_RST_PANIC: Serial.println("[SYS] Reset: PANIC"); break;
        case ESP_RST_INT_WDT: Serial.println("[SYS] Reset: INT WDT"); break;
        case ESP_RST_TASK_WDT: Serial.println("[SYS] Reset: TASK WDT"); break;
        case ESP_RST_BROWNOUT: Serial.println("[SYS] Reset: BROWNOUT"); break;
        default: Serial.printf("[SYS] Reset: UNKNOWN (%d)\n", reason); break;
    }
    Serial.printf("[SYS] Free heap: %u bytes\n", ESP.getFreeHeap());
    Serial.println("[SYS] =====");
}
```

```
// ===== ФОРМАТИРОВАНИЕ ДАТЫ =====
```

```
String formatDate(DateTime dt) {
    char buffer[20];
    sprintf(buffer, "%02d.%02d.%04d %02d:%02d:%02d",
        dt.day(), dt.month(), dt.year(),
        dt.hour(), dt.minute(), dt.second());
    return String(buffer);
}
```

```
float parseSafeFloat(String val) {
    val.replace(',', '.');
    val.trim();
    return val.toFloat();
}
```

```
// ===== НАПРАВЛЕНИЕ ВЕТРА =====
```

```
String getWindDirectionText(int degrees) {
    const char* directions[] = {"С", "ССВ", "СВ", "BCB", "В", "ВЮВ", "ЮВ", "ЮЮВ",
        "Ю", "ЮЮЗ", "ЮЗ", "ЗЮЗ", "З", "ЗСЗ", "СЗ", "ССЗ"};
    int idx = ((int)((degrees + 11) / 22.5)) % 16;
    return String(directions[idx]);
}
```

```

void updateWindAverage(int degrees) {
    float rad = degrees * M_PI / 180.0;
    wind_dir_history_sin[wind_sample_idx] = sin(rad);
    wind_dir_history_cos[wind_sample_idx] = cos(rad);
    wind_sample_idx++;
    if (wind_sample_idx >= WIND_AVG_SAMPLES) {
        wind_sample_idx = 0;
        wind_history_filled = true;
    }
    int count = wind_history_filled ? WIND_AVG_SAMPLES : wind_sample_idx;
    float avg_sin = 0, avg_cos = 0;
    for (int i = 0; i < count; i++) {
        avg_sin += wind_dir_history_sin[i];
        avg_cos += wind_dir_history_cos[i];
    }
    avg_sin /= count;
    avg_cos /= count;
    float avg_rad = atan2(avg_sin, avg_cos);
    wind_dir_avg = (int)(avg_rad * 180.0 / M_PI);
    if (wind_dir_avg < 0) wind_dir_avg += 360;
}

```

```

void updateWindSpeedAverage(float current_speed, float current_gust) {
    wind_speed_history[wind_speed_idx] = current_speed;
    wind_speed_idx++;
    if (wind_speed_idx >= WIND_SPEED_SAMPLES) {
        wind_speed_idx = 0;
        wind_speed_filled = true;
    }
    float sum = 0;
    int count = wind_speed_filled ? WIND_SPEED_SAMPLES : wind_speed_idx;
    for (int i = 0; i < count; i++) sum += wind_speed_history[i];
    wind_avg_smooth = sum / count;
    wind_gust_max = current_gust;
}

```

// ===== ИСТОРИЯ =====

```

void addHistory(float t, float p, float w, float r) {
    history.temp[history.head] = t;
    history.pressure[history.head] = p;
    history.wind[history.head] = w;
    history.rain[history.head] = r;
    history.head = (history.head + 1) % POINTS_HISTORY;
}

```

```

}

// ===== ZAMBRETTI =====
void updatePressureHistory(float currentPressure) {
    pressure_history[pressure_history_idx] = currentPressure;
    pressure_history_idx = (pressure_history_idx + 1) % PRESSURE_HISTORY_SIZE;
    if (pressure_history_idx == 0) pressure_history_full = true;
}

float calculatePressureTrend() {
    if (!pressure_history_full) return 0.0;
    int firstIdx = pressure_history_idx;
    int lastIdx = (pressure_history_idx - 1 + PRESSURE_HISTORY_SIZE) % PRESSURE_HISTORY_SIZE;
    return pressure_history[lastIdx] - pressure_history[firstIdx];
}

void calculateZambrettiForecast() {
    float trend = calculatePressureTrend();
    if (trend > 1.5) pressure_trend_desc = "Резкий подъём ↑ ↑ ↑";
    else if (trend > 0.5) pressure_trend_desc = "Подъём ↑ ↑";
    else if (trend < -1.5) pressure_trend_desc = "Резкий спад ↓ ↓ ↓";
    else if (trend < -0.5) pressure_trend_desc = "Спад ↓ ↓";
    else pressure_trend_desc = "Стабильно →";
    float p = pressure;
    if (p > 1030.0 && trend > 0.0) zambretti_forecast = "Отличная, ясная погода ☀️";
    else if (p > 1025.0) zambretti_forecast = "Ясная погода ☀️";
    else if (p > 1020.0 && trend < -0.5) zambretti_forecast = "Переменная облачность 🌤️";
    else if (p > 1020.0) zambretti_forecast = "Хорошая погода ☀️";
    else if (p > 1015.0 && trend > 0.0) zambretti_forecast = "Улучшение погоды 🌤️";
    else if (p > 1015.0) zambretti_forecast = "Облачно с прояснениями 🌤️";
    else if (p > 1010.0 && trend < -0.5) zambretti_forecast = "Вероятны осадки 🌧️";
    else if (p > 1010.0) zambretti_forecast = "Облачно ☁️";
    else if (p > 1005.0 && trend < -0.5) zambretti_forecast = "Дождливая погода 🌧️";
    else if (p > 1005.0) zambretti_forecast = "Пасмурно ☁️";
    else if (p > 1000.0) zambretti_forecast = "Дожди 🌧️";
    else zambretti_forecast = "Штормовое предупреждение ⚡️";
}

// ===== СКАНЕР ЧАСТОТЫ =====
float runHardwareScanner() {
    Serial.println("[CORE 0] Starting hardware frequency calibration...");
    float best_freq = current_freq;
    float max_rssi = -150.0;
    for (int step = 0; step < SCAN_STEPS; step++) {

```



```

float freq = SCAN_START_FREQ + (step * SCAN_STEP);
radio.setFrequency(freq);
radio.startReceive();
vTaskDelay(pdMS_TO_TICKS(200));
float rssi = radio.getRSSI();
Serial.printf("[SCAN] %.3f MHz -> RSSI: %.1f dBm\n", freq, rssi);
if (rssi > max_rssi) {
    max_rssi = rssi;
    best_freq = freq;
}
}
Serial.printf("[SCAN] Best frequency: %.3f MHz (RSSI: %.1f dBm)\n", best_freq, max_rssi);
return best_freq;
}

void updateAutoScanner() {
    if (!scan_active) return;
    if (scan_step_start == 0) {
        scan_step_start = millis();
        float freq = SCAN_START_FREQ + (scan_current_step * SCAN_STEP);
        radio.setFrequency(freq);
        radio.startReceive();
        vTaskDelay(pdMS_TO_TICKS(50));
        scan_step_max_rssi = radio.getRSSI();
        scan_instant_rssi = scan_step_max_rssi;
        Serial.printf("[SCAN] Step %d: %.3f MHz, RSSI=%.1f\n", scan_current_step, freq, scan_step_max_rssi);
    }
    if (millis() - scan_step_start > SCAN_STEP_TIME) {
        scan_results[scan_current_step] = scan_step_max_rssi;
        if (scan_step_max_rssi > scan_best_rssi) {
            scan_best_rssi = scan_step_max_rssi;
            scan_best_freq = SCAN_START_FREQ + (scan_current_step * SCAN_STEP);
        }
        scan_current_step++;
        scan_step_start = 0;
        if (scan_current_step >= SCAN_STEPS) {
            scan_active = false;
            if (scan_best_rssi > -95.0) {
                current_freq = scan_best_freq;
                current_working_frequency = current_freq;
                radio.setFrequency(current_freq);
                saveAll();
                Serial.printf("[SCAN] Optimal frequency: %.3f MHz\n", current_freq);
            }
        }
    }
}

```

```

rf.enableReceiver();
}
}
}

// ===== RTL433 CALLBACK (НА ЯДРЕ 0 — ТОЛЬКО ОТПРАВКА В ОЧЕРЕДЬ)
=====
void rtl433_Callback(char* message) {
  StaticJsonDocument<512> doc;
  DeserializationError error = deserializeJson(doc, message);
  if (error) return;
  const char* model = doc["model"];
  if (model && strstr(model, "FineOffset")) {
    WeatherMessage msg;
    msg.station_id = doc["id"] | 0;
    // Внимание: ключ в JSON может быть "battery_ok" или "battery_OK"
    msg.battery_ok = (doc["battery_ok"] | 0) == 1;
    msg.outdoor_temp = doc["temperature_C"] | 0.0;
    msg.outdoor_humidity = doc["humidity"] | 0;
    msg.wind_dir_deg = doc["wind_dir_deg"] | 0;
    msg.wind_speed_ms = doc["wind_avg_m_s"] | 0.0;
    msg.wind_gust_ms = doc["wind_max_m_s"] | 0.0;
    msg.rain_mm = doc["rain_mm"] | 0.0;
    msg.rssi = radio.getRSSI();
    msg.current_freq = current_working_frequency;
    msg.data_valid = true;
    // Отправляем в очередь (без блокировки)
    if (weatherQueue) {
      xQueueSend(weatherQueue, &msg, 0);
    }
  }
}

// ===== ЗАДАЧА РАДИО (CORE 0) =====
void radioTask(void *pvParameters) {
  Serial.printf("[CORE 0] Radio task started on core %d\n", xPortGetCoreID());
  // Настройка пина GDO0 (важно для прерываний)
  pinMode(PIN_GDO0, INPUT);
  // Инициализация библиотеки rtl_433_ESP (она сама настраивает SPI и CC1101)
  rf.setGdo0(PIN_GDO0);
  rf.init();
  rf.setCallback(rtl433_Callback);
  rf.enableProtocol(32); // Fine Offset WH1080/WS1080
  // Сканер частоты (использует объект radio внутри библиотеки)

```

```

current_working_frequency = runHardwareScanner();
// Устанавливаем финальную частоту и запускаем приёмник
radio.setFrequency(current_working_frequency);
rf.enableReceiver();
Serial.printf("[CORE 0] Receiver ready at %.2f MHz\n", current_working_frequency);
for (;;) {
    rf.loop();
    vTaskDelay(pdMS_TO_TICKS(1));
}
}

// ===== MQTT =====

void mqttCallback(char* topic, byte* payload, unsigned int length) {}

void sendDiscovery(const char* name, const char* unit, const char* dev_cla, const char* icon) {
    if (!mqtt.connected()) return;
    String deviceId = hostname;
    String topic = "homeassistant/sensor/" + deviceId + "/" + name + "/config";
    StaticJsonDocument<512> doc;
    doc["name"] = deviceId + " " + name;
    doc["state_topic"] = ("smart/" + deviceId + "/state").c_str();
    doc["value_template"] = ("{{ value_json." + String(name) + " }}").c_str();
    doc["unique_id"] = (deviceId + "_" + name).c_str();
    if (unit && strlen(unit) > 0) doc["unit_of_measurement"] = unit;
    if (dev_cla && strlen(dev_cla) > 0) doc["device_class"] = dev_cla;
    if (icon) doc["icon"] = icon;
    JsonObject device = doc.createNestedObject("device");
    device["identifiers"] = deviceId;
    device["name"] = "Weather Gate";
    device["sw_version"] = CURRENT_VERSION;
    device["model"] = "WS1080 Gateway with BMP280";
    device["manufacturer"] = "DIY";
    String out;
    serializeJson(doc, out);
    mqtt.publish(topic.c_str(), out.c_str(), true);
}

void registerAllSensors() {
    sendDiscovery("temperature", "°C", "temperature", "mdi:thermometer");
    sendDiscovery("humidity", "%", "humidity", "mdi:water-percent");
    sendDiscovery("pressure", "hPa", "pressure", "mdi:gauge");
    sendDiscovery("pressure_mmhg", "mmHg", "pressure", "mdi:gauge");
    sendDiscovery("wind_speed", "m/s", "wind_speed", "mdi:weather-windy");
    sendDiscovery("wind_gust", "m/s", "wind_speed", "mdi:weather-windy-variant");
}

```

```

sendDiscovery("wind_direction", "", nullptr, "mdi:compass");
sendDiscovery("rain", "mm", "precipitation", "mdi:weather-rainy");
sendDiscovery("outdoor_battery", nullptr, "battery", "mdi:battery");
sendDiscovery("forecast", nullptr, nullptr, "mdi:weather-partly-cloudy");
sendDiscovery("rssi", "dBm", "signal_strength", "mdi:signal");
sendDiscovery("freq", "MHz", nullptr, "mdi:radio-tower");
sendDiscovery("bmp_temperature", "°C", "temperature", "mdi:thermometer");
sendDiscovery("bmp_pressure", "hPa", "pressure", "mdi:gauge");
sendDiscovery("bmp_pressure_mmhg", "mmHg", "pressure", "mdi:gauge");
Serial.println("[MQTT] All sensors registered");
}

```

```

void publishMQTTData() {
if (!mqtt.connected()) return;
StaticJsonDocument<1024> doc;
doc["temperature"] = temperature;
doc["humidity"] = humidity;
doc["pressure"] = pressure;
doc["pressure_mmhg"] = pressure * 0.750062;
doc["wind_speed"] = wind_avg_smooth;
doc["wind_gust"] = wind_gust_max;
doc["wind_direction"] = wind_dir_avg;
doc["rain"] = rain;
doc["outdoor_battery"] = outdoor_battery;
doc["sensor_id"] = sensor_id;
doc["forecast"] = zambretti_forecast;
doc["rssi"] = last_rssi;
doc["freq"] = current_freq;
doc["bmp_temperature"] = bmp_temperature;
doc["bmp_pressure"] = bmp_pressure_raw + bmp_calibration_offset;
doc["bmp_pressure_mmhg"] = (bmp_pressure_raw + bmp_calibration_offset) * 0.750062;
doc["radio_errors"] = radio_errors;
doc["radio_success"] = radio_success;
doc["uptime"] = millis() / 1000;
if (rtc.begin(&I2C_Clock)) {
doc["datetime"] = formatDate(rtc.now());
}
String out;
serializeJson(doc, out);
mqtt.publish(("smart/" + hostname + "/state").c_str(), out.c_str(), false);
}

```

```

void broadcastMQTTData() {
// Форсированная отправка MQTT после получения нового пакета

```

```

if (mqtt.connected()) {
    publishMQTTData();
    lastMQTTPublish = millis();
}
}

void reconnectMQTT() {
    if (mqtt_ip.length() < 7) return;
    String clientId = "WeatherGate_" + hostname + "_" + String(ESP.getEfuseMac(), HEX);
    if (mqtt.connect(clientId.c_str(), mqtt_user.c_str(), mqtt_password.c_str())) {
        Serial.println("[MQTT] Connected to broker");
        mqtt_connected = true;
        if (!discovery_sent) {
            registerAllSensors();
            discovery_sent = true;
        }
    } else {
        Serial.printf("[MQTT] Failed, rc=%d\n", mqtt.state());
        mqtt_connected = false;
    }
}

// ===== OTA =====
bool getUpdateInfo(UpdateInfo &info) {
    if (mqtt_ip.length() < 7) return false;
    String url = "http://" + mqtt_ip + ":" + String(HA_PORT) + HA_OTA_PATH + "version.json";
    HTTPClient http;
    http.setTimeout(5000);
    if (!http.begin(url)) return false;
    int httpCode = http.GET();
    if (httpCode != HTTP_CODE_OK) {
        http.end();
        return false;
    }
    String payload = http.getString();
    http.end();
    StaticJsonDocument<768> doc;
    DeserializationError error = deserializeJson(doc, payload);
    if (error) return false;
    info.version = doc["version"] | "";
    info.fw_md5 = doc["fw_md5"] | "";
    info.fs_md5 = doc["fs_md5"] | "0";
    info.changelog = doc["changelog"] | "";
    String baseUrl = "http://" + mqtt_ip + ":" + String(HA_PORT) + HA_OTA_PATH;

```

```
info.fw_url = baseUrl + "firmware.bin";
info.fs_url = baseUrl + "littelfs.bin";
return info.version.length() > 0;
}
```

```
int compareVersions(String newVer, String curVer) {
if (newVer == curVer) return 0;
int newMajor = 0, newMinor = 0, newPatch = 0;
int curMajor = 0, curMinor = 0, curPatch = 0;
sscanf(newVer.c_str(), "%d.%d.%d", &newMajor, &newMinor, &newPatch);
sscanf(curVer.c_str(), "%d.%d.%d", &curMajor, &curMinor, &curPatch);
if (newMajor > curMajor) return 1;
if (newMajor < curMajor) return -1;
if (newMinor > curMinor) return 1;
if (newMinor < curMinor) return -1;
if (newPatch > curPatch) return 1;
if (newPatch < curPatch) return -1;
return 0;
}
```

```
void performOTA() {
Serial.println("[OTA] === OTA START ===");
UpdateInfo info;
if (!getUpdateInfo(info)) {
Serial.println("[OTA] Failed to get update info");
return;
}
if (info.version == CURRENT_VERSION) {
Serial.println("[OTA] Already up to date");
return;
}
Serial.printf("[OTA] Updating to %s\n", info.version.c_str());
mqtt.disconnect();
delay(500);
WiFiClient otaClient;
otaClient.setTimeout(15000);
if (info.fs_md5.length() > 2 && info.fs_md5 != "0") {
Serial.println("[OTA] Updating LittleFS...");
httpUpdate.setMD5sum(info.fs_md5);
httpUpdate.updateFs(otaClient, info.fs_url);
delay(500);
}
Serial.println("[OTA] Updating firmware...");
httpUpdate.setMD5sum(info.fw_md5);
}
```

```
httpUpdate.rebootOnUpdate(true);
httpUpdate.update(otaClient, info.fw_url);
}
```

```
// ===== СОХРАНЕНИЕ =====
```

```
void periodicSave() {
if (millis() - lastSaveTime >= SAVE_INTERVAL) {
saveCounters();
lastSaveTime = millis();
Serial.println("[SYS] Periodic save");
}
}
```

```
void loadSettings() {
pref.begin("weather", true);
hostname = pref.getString("hostname", "weather_gate");
if (hostname.length() == 0) hostname = "weather_gate";
mqtt_ip = pref.getString("mq_ip", "");
mqtt_user = pref.getString("mq_u", "");
mqtt_password = pref.getString("mq_p", "");
mqtt_topic_weather = pref.getString("mqtt_topic_weather", MQTT_TOPIC_WEATHER_DEFAULT);
current_freq = pref.getFloat("freq", 915.0);
bmp_calibration_offset = pref.getFloat("bmp_offset", 31.7);
altitude_meters = pref.getInt("altitude", 150);
radio_errors = pref.getUInt("radio_errors", 0);
radio_success = pref.getUInt("radio_success", 0);
temperature = pref.getFloat("temp", 0);
humidity = pref.getFloat("hum", 0);
pressure = pref.getFloat("pres", 1013.25);
temp_min = pref.getFloat("temp_min", 0);
temp_max = pref.getFloat("temp_max", 0);
pressure_min = pref.getFloat("pres_min", 1013.25);
pressure_max = pref.getFloat("pres_max", 1013.25);
wind_max = pref.getFloat("wind_max", 0);
rain = pref.getFloat("rain", 0);
rain_max = pref.getFloat("rain_max", 0);
sensor_id = pref.getInt("sensor_id", -1);
pref.end();
Serial.printf("[SYS] Settings loaded: freq=%.2f, alt=%d, offset=%.1f\n",
current_freq, altitude_meters, bmp_calibration_offset);
}
```

```
void saveAll() {
pref.begin("weather", false);
```

```

pref.putString("hostname", hostname);
pref.putString("mq_ip", mqtt_ip);
pref.putString("mq_u", mqtt_user);
pref.putString("mq_p", mqtt_password);
pref.putString("mqtt_topic_weather", mqtt_topic_weather);
pref.putFloat("freq", current_freq);
pref.putFloat("bmp_offset", bmp_calibration_offset);
pref.putInt("altitude", altitude_meters);
pref.putUInt("radio_errors", radio_errors);
pref.putUInt("radio_success", radio_success);
pref.putFloat("temp", temperature);
pref.putFloat("hum", humidity);
pref.putFloat("pres", pressure);
pref.putFloat("rain", rain);
pref.putFloat("rain_max", rain_max);
pref.putFloat("temp_min", temp_min);
pref.putFloat("temp_max", temp_max);
pref.putFloat("pressure_min", pressure_min);
pref.putFloat("pressure_max", pressure_max);
pref.putFloat("wind_max", wind_max);
pref.putInt("sensor_id", sensor_id);
pref.end();
Serial.println("[SYS] All settings saved");
}

```

```

void saveCounters() {
pref.begin("weather", false);
pref.putUInt("radio_errors", radio_errors);
pref.putUInt("radio_success", radio_success);
pref.putFloat("temp", temperature);
pref.putFloat("hum", humidity);
pref.putFloat("pres", pressure);
pref.putFloat("rain", rain);
pref.end();
}

```

```

// ===== GZIP ОБРАБОТЧИК =====
bool handleFileRead(String path) {
if (path.endsWith("/")) path += "index.html";
String contentType = "text/plain";
if (path.endsWith(".html")) contentType = "text/html";
else if (path.endsWith(".js")) contentType = "application/javascript";
else if (path.endsWith(".css")) contentType = "text/css";
else if (path.endsWith(".ico")) contentType = "image/x-icon";

```



```

String gzPath = path + ".gz";
if (LittleFS.exists(gzPath)) {
File file = LittleFS.open(gzPath, "r");
if (file) {
server.setContentLength(file.size());
server.setHeader("Content-Encoding", "gzip");
server.setHeader("Cache-Control", "public, max-age=86400");
server.send(200, contentType, "");
server.client().write(file);
file.close();
return true;
}
}
if (LittleFS.exists(path)) {
File file = LittleFS.open(path, "r");
if (file) {
server.streamFile(file, contentType);
file.close();
return true;
}
}
return false;
}

// ===== BMP280 =====
void readBMP280() {
if (!bmp.begin(0x76, &I2C_Bmp)) return;
float p_raw = bmp.readPressure() / 100.0F;
if (p_raw < 850.0 || p_raw > 1100.0) return;
bmp_pressure_raw = p_raw;
bmp_temperature = bmp.readTemperature();
float calibrated = bmp_pressure_raw + bmp_calibration_offset;
pressure = calibrated + (altitude_meters * 0.12);
static unsigned long lastBMPLog = 0;
if (millis() - lastBMPLog > 60000) {
Serial.printf("[BMP280] P=%.1f hPa, T=%.1f°C\n", pressure, bmp_temperature);
lastBMPLog = millis();
}
}

// ===== СЕТЕВЫЕ СОБЫТИЯ =====
void WiFiEvent(WiFiEvent_t event) {
switch (event) {
case ARDUINO_EVENT_ETH_START:

```

```

Serial.println("[ETH] Ethernet Started");
ETH.setHostname(hostname.c_str());
break;
case ARDUINO_EVENT_ETH_CONNECTED:
Serial.println("[ETH] Ethernet Connected");
break;
case ARDUINO_EVENT_ETH_GOT_IP:
Serial.printf("[ETH] Got IP: %s\n", ETH.localIP().toString().c_str());
eth_connected = true;
break;
case ARDUINO_EVENT_ETH_DISCONNECTED:
Serial.println("[ETH] Ethernet Disconnected");
eth_connected = false;
break;
default:
break;
}
}

// ===== SETUP =====
void setup() {
Serial.begin(115200);
delay(500);
printResetReason();
// Отключаем ненужные светодиоды
pinMode(17, INPUT); // GPIO17 свободен, отключаем LED
// I2C Канал 0 (часы DS3231)
I2C_Clock.begin(PIN_I2C0_SDA, PIN_I2C0_SCL, 100000);
// I2C Канал 1 (барометр BMP280)
I2C_Bmp.begin(PIN_I2C1_SDA, PIN_I2C1_SCL, 100000);
// LittleFS
if (!LittleFS.begin(true)) {
Serial.println("[FS] LittleFS mount failed");
} else {
Serial.println("[FS] LittleFS mounted");
}
// Загрузка настроек
loadSettings();
// Ethernet
WiFi.mode(WIFI_OFF);
btStop();
WiFi.onEvent(WiFiEvent);
ETH.begin(ETH_PHY_LAN8720, 1, 16, 23, 18, ETH_PHY_LAN8720);
delay(500);

```

```

// MQTT
if (mqtt_ip.length() > 5) {
  mqtt.setServer(mqtt_ip.c_str(), 1883);
  mqtt.setCallback(mqttCallback);
  mqtt.setKeepAlive(60);
}
// Очередь FreeRTOS
weatherQueue = xQueueCreate(10, sizeof(WeatherMessage));
if (weatherQueue == NULL) {
  Serial.println("[SYS] ERROR: Failed to create queue!");
}
// Запуск задачи радио на Core 0 (высокий приоритет)
xTaskCreatePinnedToCore(
  radioTask,
  "RadioTask",
  10240,
  NULL,
  3,
  NULL,
  0
);
// Инициализация истории
for (int i = 0; i < POINTS_HISTORY; i++) {
  history.temp[i] = temperature;
  history.pressure[i] = pressure;
  history.wind[i] = 0;
  history.rain[i] = 0;
}
for (int i = 0; i < PRESSURE_HISTORY_SIZE; i++) {
  pressure_history[i] = pressure;
}
// Инициализация массивов усреднения ветра
for (int i = 0; i < WIND_AVG_SAMPLES; i++) {
  wind_dir_history_sin[i] = 0;
  wind_dir_history_cos[i] = 0;
  wind_speed_history[i] = 0;
}
// ===== WEB-СЕРВЕР =====
// Текущие данные
server.on("/api/data", HTTP_GET, []() {
  StaticJsonDocument<2048> doc;
  doc["temperature"] = temperature;
  doc["humidity"] = humidity;
  doc["pressure"] = pressure;

```

```

doc["pressure_mmhg"] = pressure * 0.750062;
doc["wind_speed"] = wind_avg_smooth;
doc["wind_gust"] = wind_gust_max;
doc["wind_direction"] = wind_dir_avg;
doc["wind_dir_txt"] = getWindDirectionText(wind_dir_avg);
doc["rain"] = rain;
doc["rain_max"] = rain_max;
doc["temp_min"] = temp_min;
doc["temp_max"] = temp_max;
doc["pressure_min"] = pressure_min;
doc["pressure_max"] = pressure_max;
doc["wind_max"] = wind_max;
doc["outdoor_battery"] = outdoor_battery;
doc["sensor_id"] = sensor_id;
doc["mqtt_status"] = mqtt.connected() ? "OK" : "Offline";
doc["hostname"] = hostname;
doc["version"] = CURRENT_VERSION;
doc["uptime"] = millis() / 1000;
doc["radio_errors"] = radio_errors;
doc["radio_success"] = radio_success;
doc["current_freq"] = current_freq;
doc["forecast"] = zambretti_forecast;
doc["pressure_trend"] = pressure_trend_desc;
doc["rssi"] = last_rssi;
doc["bmp_temperature"] = bmp_temperature;
doc["bmp_pressure"] = bmp_pressure_raw + bmp_calibration_offset;
doc["bmp_pressure_mmhg"] = (bmp_pressure_raw + bmp_calibration_offset) * 0.750062;
doc["altitude"] = altitude_meters;
doc["bmp_offset"] = bmp_calibration_offset;
if (rtc.begin(&I2C_Clock)) {
doc["datetime"] = formatDate(rtc.now());
}
String buf;
serializeJson(doc, buf);
server.send(200, "application/json", buf);
});
// История
server.on("/api/history", HTTP_GET, []() {
StaticJsonDocument<6144> doc;
JSONArray tempHist = doc.createNestedArray("temp_history");
JSONArray pressureHist = doc.createNestedArray("pressure_history");
JSONArray windHist = doc.createNestedArray("wind_history");
JSONArray rainHist = doc.createNestedArray("rain_history");
for (int i = 0; i < POINTS_HISTORY; i++) {

```

```

int idx = (history.head + i) % POINTS_HISTORY;
tempHist.add(history.temp[idx]);
pressureHist.add(history.pressure[idx]);
windHist.add(history.wind[idx]);
rainHist.add(history.rain[idx]);
}
String buf;
serializeJson(doc, buf);
server.send(200, "application/json", buf);
});
// Синхронизация времени
server.on("/api/sync_time", HTTP_POST, []() {
if (server.hasArg("t") && rtc.begin(&I2C_Clock)) {
rtc.adjust(DateTime(server.arg("t").toInt()));
server.send(200, "text/plain", "OK");
} else {
server.send(400, "text/plain", "Failed");
}
});
// Установка частоты
server.on("/api/set_freq", HTTP_POST, []() {
if (server.hasArg("f")) {
float new_freq = server.arg("f").toFloat();
if (new_freq >= 860.0 && new_freq <= 930.0) {
current_freq = new_freq;
current_working_frequency = new_freq;
radio.setFrequency(current_freq);
saveAll();
server.send(200, "text/plain", "OK");
} else {
server.send(400, "text/plain", "Invalid frequency");
}
}
});
// Сброс ошибок
server.on("/api/reset_errors", HTTP_POST, []() {
radio_errors = 0;
radio_success = 0;
saveCounters();
server.send(200, "text/plain", "OK");
});
// Проверка обновлений
server.on("/api/check_update", HTTP_GET, []() {
StaticJsonDocument<512> doc;

```

```

UpdateInfo info;
if (getUpdateInfo(info)) {
    doc["has_update"] = (info.version != CURRENT_VERSION);
    doc["current_version"] = CURRENT_VERSION;
    doc["new_version"] = info.version;
    doc["changelog"] = info.changelog;
} else {
    doc["has_update"] = false;
    doc["error"] = "Failed to fetch update info";
}
String buf;
serializeJson(doc, buf);
server.send(200, "application/json", buf);
});

// Запуск обновления
server.on("/api/do_update", HTTP_POST, []() {
    static unsigned long lastOTA = 0;
    if (millis() - lastOTA < 60000) {
        server.send(429, "text/plain", "Too frequent requests");
        return;
    }
    lastOTA = millis();
    server.send(200, "text/plain", "Update started...");
    delay(500);
    performOTA();
});

// Запуск сканера
server.on("/api/start_scan", HTTP_GET, []() {
    if (!scan_active) {
        scan_active = true;
        scan_current_step = 0;
        scan_best_rssi = -150.0;
        scan_step_start = 0;
        for (int i = 0; i < SCAN_STEPS; i++) scan_results[i] = -120.0;
        server.send(200, "text/plain", "OK");
    } else {
        server.send(409, "text/plain", "Scan already active");
    }
});

// Данные сканера
server.on("/api/scan_data", HTTP_GET, []() {
    StaticJsonDocument<1024> doc;
    doc["active"] = scan_active;
    doc["step"] = scan_current_step;

```

```

doc["instant"] = scan_instant_rssi;
doc["rssi_current"] = radio.getRSSI();
JSONArray data = doc.createNestedArray("rssi");
for (int i = 0; i < SCAN_STEPS; i++) data.add(scan_results[i]);
String buf;
serializeJson(doc, buf);
server.send(200, "application/json", buf);
});
// Сохранение настроек
server.on("/api/config", HTTP_POST, []() {
String body = server.arg("plain");
StaticJsonDocument<2048> doc;
DeserializationError error = deserializeJson(doc, body);
if (error) {
server.send(400, "text/plain", "JSON Error");
return;
}
if (doc.containsKey("hostname")) hostname = doc["hostname"].as<String>();
if (doc.containsKey("mq_ip")) mqtt_ip = doc["mq_ip"].as<String>();
if (doc.containsKey("mq_u")) mqtt_user = doc["mq_u"].as<String>();
if (doc.containsKey("mq_p")) mqtt_password = doc["mq_p"].as<String>();
if (doc.containsKey("mqtt_topic_weather")) mqtt_topic_weather = doc["mqtt_topic_weather"].as<String>();
if (doc.containsKey("altitude")) altitude_meters = doc["altitude"].as<int>();
if (doc.containsKey("bmp_offset")) bmp_calibration_offset =
parseSafeFloat(doc["bmp_offset"].as<String>());
if (hostname.length() > 0) ETH.setHostname(hostname.c_str());
mqtt.setServer(mqtt_ip.c_str(), 1883);
saveAll();
server.send(200, "OK");
delay(500);
ESP.restart();
});
server.on("/", HTTP_GET, []() {
if (!handleFileRead("/")) {
server.send(404, "text/plain", "index.html not found");
}
});
server.onNotFound([]() {
if (!handleFileRead(server.uri())) {
server.send(404, "text/plain", "Not Found");
}
});
server.begin();
Serial.println("[HTTP] Server started");

```

```

lastSaveTime = millis();
lastHistoryTime = millis();
lastBMPRead = millis();
// Первое чтение BMP280
readBMP280();
Serial.println("[SYS] Setup complete");
Serial.printf("[SYS] Waiting for WS1080 packets at %.2f MHz\n", current_freq);
}

// ===== LOOP (CORE 1) =====
void loop() {
server.handleClient();
static uint32_t lastBMPCheck = millis();
static uint32_t lastMQTTCheck = millis();
// Опрос BMP280 (раз в 30 секунд)
if (millis() - lastBMPCheck >= BMP_READ_INTERVAL) {
lastBMPCheck = millis();
readBMP280();
}
// ===== ОСНОВНОЕ: ПРИЁМ ДАННЫХ ИЗ ОЧЕРЕДИ =====
WeatherMessage msg;
if (weatherQueue && xQueueReceive(weatherQueue, &msg, 0) == pdPASS) {
// Копируем данные в глобальные переменные (безопасно на Core 1)
temperature = msg.outdoor_temp;
humidity = msg.outdoor_humidity;
wind_avg_smooth = msg.wind_speed_ms;
wind_gust_max = msg.wind_gust_ms;
wind_dir_avg = msg.wind_dir_deg;
rain = msg.rain_mm;
sensor_id = msg.station_id;
last_rssi = msg.rssi;
current_freq = msg.current_freq;
outdoor_battery = msg.battery_ok ? "OK" : "LOW";
radio_success++;
// Обновление статистики
if (temperature < temp_min || temp_min == 0) temp_min = temperature;
if (temperature > temp_max) temp_max = temperature;
if (wind_avg_smooth > wind_max) wind_max = wind_avg_smooth;
if (rain > rain_max) rain_max = rain;
// Усреднение ветра (векторное)
updateWindAverage(wind_dir_avg);
updateWindSpeedAverage(wind_avg_smooth, wind_gust_max);
// Лог в монитор порта
static float last_log_temp = 0;

```



```

if (fabs(temperature - last_log_temp) > 0.3 || last_log_temp == 0) {
    Serial.printf("[WS1080] T=%.1f H=%.0f%% W=%.1f D=%d° Rain=%.1f Batt=%s RSSI=%.0f Freq=%.2f\n",
        temperature, humidity, wind_avg_smooth, wind_dir_avg, rain,
        outdoor_battery.c_str(), last_rssi, current_freq);
    last_log_temp = temperature;
}
// Немедленная отправка в MQTT (свежие данные)
if (mqtt.connected()) {
    broadcastMQTTData();
}
}
// ===== MQTT ОБСЛУЖИВАНИЕ =====
if (eth_connected) {
    if (mqtt.connected()) {
        mqtt.loop();
    } else if (millis() - lastMQTTReconnect >= MQTT_RECONNECT_INTERVAL) {
        lastMQTTReconnect = millis();
        reconnectMQTT();
    }
}
// ===== ПЕРИОДИЧЕСКИЕ ЗАДАЧИ =====
static uint32_t lastPeriodic = millis();
if (millis() - lastPeriodic >= 10000) {
    lastPeriodic = millis();
    periodicSave();
}
// История и прогноз (раз в 5 минут, при наличии новых данных)
static uint32_t lastHistory = millis();
if (millis() - lastHistory >= HISTORY_INTERVAL && radio_success > 0) {
    lastHistory = millis();
    addHistory(temperature, pressure, wind_avg_smooth, rain);
    updatePressureHistory(pressure);
    calculateZambrettiForecast();
}
// Сканер
if (scan_active) {
    updateAutoScanner();
}
delay(1);
}

```